

Forge beyond Universal Closure

Alternating strategies, probabilistic couplings,
and quantitative liveness

A repository-relative design with exact executable prototypes

15 September 2026

Abstract

Forge already proposes a broad certificate-producing layer around Lean’s `grind`, including induction, arithmetic witnesses, nonlinear certificates, and finite closures. Adding those ideas again would not extend it. This article instead develops four compatible workers for obligations in which choices are adversarial, executions are infinite, or transitions are probabilistic: parity-game strategy certificates; optimal finite couplings with Hall-dual obstructions; alternating probabilistic simulation; and worst-case expected termination with Bellman potentials and reachable nontermination traps.

The contribution is an integration design and a set of independently replayable certificate formats, not a claim to have invented these mathematical subjects. The article proves the local soundness arguments, states the exact finite fragments for which the algorithms are complete, and identifies the source-to-Lean bridges still required. The accompanying standard-library-only Python implementation was executed against independent finite oracles: 884 parity arenas, 400 transport problems, 180 Markov decision processes, and 100 simulation problems. Fourteen semantic regressions pass, every deliberately invalid mutation is rejected, and all stored certificates replay with search imports forbidden. These are Python results, not Lean-kernel proofs or a benchmark against existing tactics.

Reading the evidence. The algorithms and Python checkers are implemented and tested. The Lean lowering specimens are uncompiled. No new Lean `forge` tactic, source reifier, or formally verified certificate checker is claimed. The proposed extension is relative to Forge commit `c98e47c5f804e92880fc1d0e378c1b95832b685c`.

Contents

1	The extension boundary	4
1.1	What the inspected repository already contains	4
1.2	Why these are related capabilities	4
1.3	A concrete target workload	5
1.4	What was actually built	5
1.5	Pinned context and scope of external tools	6
2	Alternating infinite behavior: parity strategies	6
2.1	The source of the additional power	6
2.2	A local certificate with no trusted game solver	6
2.3	Concrete search algorithm	7
2.4	Two semantic regression examples	8
2.5	How the strategy becomes a Lean witness	8
3	Relational probability: exact couplings with an optimality dual	9
3.1	Why independent execution is the wrong default	9
3.2	The Hall lower bound	9
3.3	An exact proposer	10
3.4	Worked examples	11
3.5	Composition with finite probabilistic programs	11
3.6	A precise next extension: transport costs	12
4	Alternating probabilistic simulation	12
4.1	A fixed point whose quantifiers matter	12
4.2	Positive and negative evidence in one certificate	13
4.3	Algorithm, complexity, and incremental improvement	13
4.4	A quantifier-order example	14
4.5	The exact meaning of a negative answer	14
4.6	Connection to Leant and Djex	14
5	Quantitative liveness: expected termination under every scheduler	15
5.1	A graph cycle need not prevent probabilistic termination	15
5.2	A local Bellman certificate	15
5.3	A finite certificate of nontermination	16
5.4	A complete algorithm on the stated finite fragment	17
5.5	Worked scheduler examples	18
5.6	A symbolic extension that reuses existing Forge machinery	18

6	Integration into Forge and Lean	19
6.1	Extend the workers, not the architecture	19
6.2	A small neutral problem language	19
6.3	Source bridges as the critical new obligations	20
6.4	Proposed module boundaries	20
6.5	Result types should name their authority	21
6.6	Concrete library choices	21
6.7	How to use <code>grind</code> without treating it as a model checker	22
7	Implementation and experimental evidence	22
7.1	Execution conditions and units	22
7.2	Parity: exhaustive small arenas and random larger ones	23
7.3	Transport: exact optima and an actual ablation	23
7.4	Expected time: a structurally different oracle	24
7.5	Simulation: enumerate relations, not elimination traces	24
7.6	Replay without search or test oracles	24
7.7	What the measured timings do and do not mean	25
7.8	Remaining implementation weaknesses	25
8	Implementation sequence and acceptance gates	25
8.1	First ship a probability bridge, not four new tactic names	25
8.2	Six concrete delivery gates	26
8.3	A useful composite target	26
8.4	Evaluation after Lean integration	27
8.5	Research directions after the implemented fragment	27
8.6	Assessment	27
A	Artifact guide and reproducibility	28
A.1	Archive layout	28
A.2	Run the experiments	28
A.3	Build the PDF	29
A.4	Reviewed source snapshots	29
A.5	Certificate fields and checker costs	29
A.6	The included Lean specimens	30

1 The extension boundary

1.1 What the inspected repository already contains

The inspected Forge snapshot is not an empty proposal. Its merged article is 111 pages, draws on eighteen preserved submissions, and already contains the usual candidates for a stronger proof planner. Its canonical status ledger, provenance appendix, conclusion, and Leant/Djex review were used to delimit this work [1, 2, 3, 4]. In particular, induction with generalization, arithmetic witness programs, ideal closure, finite-algebra covers, observable-space closure, Kripke countermodels, nonlinear certificates, boundary-safe telescoping, and cyclic descent are *prerequisites*, not contributions of this article.

The merge also records successful elaboration of sixteen core-only Lean files. It would therefore be wrong to say that nothing in Forge has ever compiled. The relevant remaining gap is narrower: the canonical ledger does not claim an implemented `forge` tactic or a Lean-verified certificate checker for its workers [2]. This article does not close that gap merely by adding Python experiments.

The non-duplication claim here is scoped to the canonical merged design and capability inventory at the pinned commit. It is not a claim that every line of every historical submission has been exhaustively searched, nor that the algorithms below are new to the literature. The repository itself encourages a small checked implementation over another repetition of its architecture; the proposal below keeps that architecture and changes the kinds of obligations its workers can settle.

Existing mechanism	What it does not itself provide	Addition in this article
Universal word/observable closure	A controller’s choices against an adversary, with an infinite recurrence condition	Winning positional strategies plus threshold-local parity ranks
Finite-state or finite-algebra closure	Correlated executions with prescribed probability marginals	Full rational couplings and optimal unsupported-mass certificates
Closure under all transitions	The alternating order “for every source action, there exists a matching target action”	Greatest probabilistic simulation with replayable elimination evidence
Well-founded cyclic descent	A loop that may repeat forever as a graph path, but terminates almost surely	Bellman drift certificates for expected time, or a reachable controlled trap
Typed witness synthesis	Behavioral correctness of a strategy or a coupled transition table	Source-owned strategy tables and probabilistic witnesses, with separate semantic proof obligations

1.2 Why these are related capabilities

A useful way to view the extension is to distinguish three transition operators. For a predicate P , a demonic step requires every successor to satisfy P , whereas an angelic step requires a selectable successor. For a nonnegative numerical potential V , a probabilistic step transports V by an expectation. The same proof-planning mistake can occur in all three settings: the operator is silently changed to an easier one.

Replacing $\forall a \exists b$ by $\exists b \forall a$ loses valid strategy witnesses; replacing it by $\exists a \exists b$ accepts invalid ones. Replacing an expectation by pointwise decrease loses randomized termination proofs. Replacing a

coupling by an independent product loses valid relational proofs. Treating a finite occurrence of a good priority as recurring accepts a false liveness claim. Each worker below carries enough information for its checker to distinguish precisely these cases.

The shared design principle is consequently more specific than “use another solver”: *reify the transition quantifier, propose a finite witness, and check its local obligations under the original operator*. Probability mass, choice ownership, terminal semantics, and the observation interface are part of the problem, not conveniences for a solver to reconstruct.

1.3 A concrete target workload

Four kinds of Lean goals motivate the workers. The following is schematic notation, not syntax installed by the accompanying package:

```
-- A controller exists against every legal environment strategy.
exists controller, forall environment,
  parityWins (run arena controller environment start)

-- Two probabilistic computations admit a relational joint execution.
exists joint, marginals joint = (law left, law right)
  and probability joint (not relation) <= epsilon

-- Every implementation action admits a matching specification action.
probabilisticSimulation source specification initialPair

-- Every scheduler has bounded expected termination time.
forall scheduler, expectedSteps program scheduler start <= budget
```

These are not four names for a finite reachability computation. The first is an infinite-game statement. The second has a joint-distribution witness. The third has alternating choices before sampling. The fourth quantifies over schedulers and estimates a numerical random variable. The finite input fragments let each worker return a small certificate even though the semantic claim concerns arbitrarily long executions.

1.4 What was actually built

The archive contains a new package, `forge_ap`, rather than a modified copy of Forge. Its search module implements recursive Zielonka solving, rational max-flow, a greatest-simulation elimination loop, and exact policy iteration. Its checker module performs only local graph, sum, and inequality checks; it imports no search algorithm. Separate tiny-model oracles use strategy enumeration, Hall-subset enumeration, exhaustive subrelations, and determinant expansion. This is stronger separation than replacing a solver entry point with an exception while retaining the solver’s arithmetic assembler, although the implementations still share representations and Python’s arithmetic runtime.

No compiler was available for the Lean part of this execution. Accordingly, `lean/` contains small, explicitly uncompiled lowering specimens; the experiment ledger records `NOT_RUN`. The mathematical proofs in the article explain what a future Lean checker must establish. They do not turn Python execution into kernel acceptance.

1.5 Pinned context and scope of external tools

The inspected repository revisions are recorded in Appendix A. Forge’s pinned Lean baseline is v4.34.0 with Mathlib revision 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa. Leant and Djex were consulted for the current source-owned synthesis and provider-selection surface, not to re-present the verification-boundary review Forge already contains [7, 8, 4].

Oink, Spot, and Storm are concrete candidates for scaling search, not dependencies of the delivered Python code. The mathematics of coupling-based relational verification and parity solving is established prior work [10, 11]. All adaptations and proofs below are stated directly so that neither an external solver’s verdict nor a citation is needed to check an individual certificate.

2 Alternating infinite behavior: parity strategies

2.1 The source of the additional power

Suppose a protocol must keep granting requests while an adversary chooses when to issue them. A finite-state invariant may establish that no illegal grant occurs. It does not establish that requests are eventually answered, and a termination rank is inappropriate because the intended protocol runs forever. An infinite game packages the missing alternation and recurrence information. Parity games are a standard vehicle for such reasoning; Oink provides a mature implementation of several solvers [10]. Here we deliberately separate solving from a particularly simple certificate for a supplied strategy.

Definition 2.1 (Finite max-parity arena). An arena consists of a nonempty finite set S , a total directed edge relation $E \subseteq S \times S$, an owner $o : S \rightarrow \{0, 1\}$, and priorities $p : S \rightarrow \mathbb{N}$. At a vertex owned by player i , that player selects the next edge. An infinite play is won by the parity of the *largest priority occurring infinitely often*. A positional strategy chooses a successor using only the current vertex.

The qualifier “infinitely often” and the choice of *largest*, rather than smallest, must appear in the source bridge and the serialization version. A solver using min-parity can be adapted only through an explicit, parity-preserving conversion. Dead ends are rejected: declaring them losing, winning, or self-looping would otherwise change the problem.

2.2 A local certificate with no trusted game solver

For player i , the certificate supplies a region W_i and a strategy σ_i on the vertices of W_i that it owns. The checker first establishes

$$o(v) = i, v \in W_i \implies (v, \sigma_i(v)) \in E \wedge \sigma_i(v) \in W_i, \quad (1)$$

$$o(v) \neq i, v \in W_i \implies \forall w, (v, w) \in E \Rightarrow w \in W_i. \quad (2)$$

Let H_i retain the selected edge at owned vertices and *all* edges at opponent vertices. This graph represents every play consistent with the proposed strategy. Checking only the edges the solver happened to explore would be unsound.

For every priority q occurring in W_i whose parity is unfavorable to i , form the threshold graph

$$H_{i, \leq q} = H_i[\{v \in W_i : p(v) \leq q\}].$$

The certificate gives a natural-number rank r_q on that graph. Every retained edge $v \rightarrow w$ with both priorities at most q must satisfy

$$r_q(v) \geq r_q(w) + \mathbf{1}_{\{p(v)=q\}}. \quad (3)$$

There is no condition across an edge passing through a priority larger than q . The rank may reset there. This is exactly what a single global decreasing rank cannot express.

The implementation stores one rank array per unfavorable threshold, with irrelevant entries fixed to zero and every relevant entry at most $|S|$. It also supplies both regions and checks that they partition the entire arena. For a goal about one initial state, a single suitable region is mathematically enough; the stronger partition contract makes the delivered solver's complete answer easy to test.

Theorem 2.2 (Soundness of threshold-local ranks). *If (1), (2), and (3) hold, every play starting in W_i and following σ_i is won by player i .*

Proof. The closure checks keep every vertex of the play in W_i . Since S is finite, at least one priority occurs infinitely often. Let q be the largest such priority. All priorities greater than q occur only finitely many times, so some tail of the play lies in $H_{i,\leq q}$. If q were unfavorable, its certificate rank would never increase along this tail and would strictly decrease on every departure from a q -vertex. There are infinitely many such departures. A natural number admits only finitely many strict decreases, a contradiction. Thus q has parity i . $\square$

Proposition 2.3 (Completeness of the certificate for a winning strategy). *Fix a positional strategy that wins from every vertex in a closed region W_i . For each unfavorable q , ranks satisfying (3) exist with values at most $|S|$.*

Proof. The threshold graph has no directed cycle containing a q -vertex. Otherwise an opponent could follow that cycle forever; all owned edges on it already agree with the fixed strategy. The largest recurrent priority would be q , contradicting winningness. Give each threshold edge weight $\mathbf{1}_{\{p(v)=q\}}$. Every directed cycle therefore has weight zero. The largest weight of a finite walk starting at v is finite: removing repeated-vertex cycles never changes its weight, so a maximizing walk has weight at most $|S|$. This largest weight is a rank satisfying the required inequalities. $\square$

Together with positional determinacy of finite parity games, the proposition explains why the certificate language is sufficient for complete finite-arena solving. It does *not* make the recursive search polynomial, and it is not a claim to have implemented the lexicographic small-progress-measures algorithm. The certificate is intentionally a simple post-processing object for a chosen strategy. Standard solver and progress-measure families are documented in Oink [10].

2.3 Concrete search algorithm

The proposer uses recursive Zielonka solving. In a subarena U , let q be the largest priority, let $i = q \bmod 2$, and let T contain the vertices of priority q . Compute the attractor $A = \text{Attr}_i(T, U)$: repeatedly add an i -vertex with a successor already in A , or an opponent vertex all of whose subarena successors lie in A . Store the chosen move whenever an owned vertex is added.

Solve $U \setminus A$ recursively. If the opponent's winning region is empty, player i wins U using the attractor strategy and the recursive strategy. Otherwise attract the opponent to its recursively winning region, remove this second attractor, solve the remainder, and combine the two strategies. The implementation checks that recursive subarenas remain total; a malformed recursion is a diagnostic failure, never negative evidence.

```

solve(U):
  if U is empty: return empty winning regions and strategies
  q := maximum priority in U
  i := q mod 2
  A, reachA := attract(i, vertices of priority q, U)
  W, sigma := solve(U minus A)
  if W[1-i] is empty:
    return combine(i wins A, reachA, W, sigma)
  B, reachB := attract(1-i, W[1-i], U)
  V, tau := solve(U minus B)
  return combine(1-i wins B, reachB, sigma[1-i], V, tau)

```

After strategies are found, the prototype computes rank arrays by repeated relaxation of (3), starting at zero. A nonconvergent positive cycle causes rejection. A production emitter should instead compute strongly connected components of each threshold graph and longest weighted paths in its condensation; this changes rank extraction, not checking.

Writing $n = |S|$, $m = |E|$, and d for the number of distinct unfavorable thresholds, the rank certificate occupies $O(dn \log n)$ bits apart from the strategy and region encoding. Its mathematical local checker performs $O(dm)$ rank comparisons and $O(n + m)$ closure work. The supplied Python implementation uses straightforward list/set membership rather than claiming an optimized realization of every asymptotic bound. Its rank emitter uses at most $O(dnm)$ relaxation work. Recursive Zielonka search retains its exponential worst-case behavior; an Oink adapter would be the appropriate scaling experiment.

2.4 Two semantic regression examples

Consider the forced cycle

$$a \ (p = 1) \longrightarrow b \ (p = 2) \longrightarrow a.$$

Player 0 wins because priority 2 recurs infinitely often. There is no natural rank strictly decreasing around the whole cycle. For the sole unfavorable threshold $q = 1$, the threshold graph has the vertex a but no edge, so the zero rank suffices. This is not evasion: visiting priority 2 is precisely what makes unbounded repetition acceptable.

By contrast,

$$a \ (p = 2) \longrightarrow b \ (p = 1), \quad b \longrightarrow b$$

is won by player 1. The largest priority seen anywhere is 2, but the largest one seen infinitely often is 1. Both examples are executed regressions. Another regression constructs a strategy certificate for an arena with one adversarial edge removed and verifies that it is rejected against the original arena.

2.5 How the strategy becomes a Lean witness

The source result should have the form “there exists a legal controller such that every legal environment run satisfies the parity condition.” A table σ_i is an executable witness once a proved finite enumeration maps source states and actions to the arena. The proof uses Theorem 2.2; it must not identify an arbitrary source function with a vertex table by pretty printing or by sampling its behavior.

A finite arena may be an abstraction of an infinite-state source program. Then a winning abstract controller transfers only with a proved alternating simulation in the correct direction: the controller’s

abstract choices must be realizable, and every concrete adversarial move must be covered. An opponent-winning abstract strategy need not refute concrete controller existence if the abstraction introduced spurious adversarial moves. Negative transfer consequently needs an exact model or a separately proved reverse bridge. This distinction is not encoded by a bare Boolean “abstract model verified” flag.

Spot can propose temporal automata and manipulate parity games [12]. However, checking a strategy in an automaton does not check that the automaton recognizes the original temporal formula. The first release should accept explicit, already interpreted parity arenas. A verified temporal translation can be added later without putting the translator into the trusted boundary.

3 Relational probability: exact couplings with an optimality dual

3.1 Why independent execution is the wrong default

Let two programs return fair bits. Their output distributions are equal, but two *independent* runs agree only with probability 1/2. A proof of equality in distribution must not require independent executions to return equal values. Instead, it can exhibit a joint distribution with the required marginals that puts both bits on the diagonal. Couplings are a standard foundation for such relational proofs [11]. The new worker computes them exactly and also explains when a perfect coupling is impossible.

Definition 3.1 (Finite coupling and defect). Let $\mu : X \rightarrow \mathbb{Q}_{\geq 0}$ and $\nu : Y \rightarrow \mathbb{Q}_{\geq 0}$ be probability distributions on nonempty finite sets. A coupling is a matrix $\gamma : X \times Y \rightarrow \mathbb{Q}_{\geq 0}$ with row sums μ and column sums ν . For a relation $R \subseteq X \times Y$, its defect is

$$\delta_R(\gamma) = \sum_{(x,y) \notin R} \gamma(x,y).$$

A zero-defect coupling is supported by R .

The problem supplied externally to the checker consists of the two complete marginals and the relation. The certificate supplies the *full* coupling, not only its favorable entries. Otherwise a proposer could hide missing mass and report a zero defect for two incompatible distributions.

3.2 The Hall lower bound

For $A \subseteq X$, put $R(A) = \{y : \exists x \in A, R(x,y)\}$ and define

$$h(A) = \mu(A) - \nu(R(A)).$$

The empty subset gives $h(\emptyset) = 0$.

Lemma 3.2 (Weighted Hall obstruction). *For every coupling γ and every $A \subseteq X$, $\delta_R(\gamma) \geq h(A)$.*

Proof. The mass in rows A is $\mu(A)$. At most $\nu(R(A))$ of this mass can lie in columns $R(A)$, by the column marginal equations and nonnegativity. Every entry in $A \times (Y \setminus R(A))$ is outside R . Its total mass is therefore a lower bound on the defect, proving the inequality. $\square$

A certificate contains a coupling γ , a claimed defect δ , and one subset A . The checker verifies nonnegativity, both marginals, the actual sum outside R , and

$$\delta_R(\gamma) = \delta = h(A). \tag{4}$$

No flow network is reconstructed by the checker.

Theorem 3.3 (Optimality from a primal–dual equality). *If the transport certificate is accepted, γ minimizes defect among all couplings of μ and ν , and δ is the exact minimum. In particular, $\delta > 0$ proves that no coupling supported by R exists.*

Proof. The supplied matrix attains δ , while Lemma 3.2 gives the same lower bound for every other coupling. $\square$

This lower-bound evidence is stronger than “the flow search failed to find a perfect matching.” It is also narrower than a claim about arbitrary source programs: it proves an obstruction for these marginals and this relation. The source bridge must establish that those are the ones in the original goal.

3.3 An exact proposer

Build a bipartite flow network with source s , sink t , an edge $s \rightarrow x$ of capacity $\mu(x)$, an edge $y \rightarrow t$ of capacity $\nu(y)$, and $x \rightarrow y$ of capacity 1 exactly when $R(x, y)$. A maximum flow has value f and produces a partial supported coupling g . The prototype uses Edmonds–Karp augmenting paths with Python `Fraction`; there is no scaling to an approximate integer grid.

Let residual row masses and column masses be

$$r_x = \mu(x) - \sum_y g(x, y), \quad c_y = \nu(y) - \sum_x g(x, y), \quad \delta = \sum_x r_x = \sum_y c_y = 1 - f.$$

If $\delta = 0$, take $\gamma = g$ and $A = \emptyset$. Otherwise complete the partial coupling by

$$\gamma(x, y) = g(x, y) + \frac{r_x c_y}{\delta}. \quad (5)$$

The added matrix has the missing marginals. Moreover, $r_x c_y > 0$ implies $\neg R(x, y)$: an admissible such pair would permit another positive augmentation. Since $f < 1$, its capacity-1 edge cannot already be saturated by a total flow of size less than 1. Hence the completion adds exactly δ unsupported mass.

Let A be the left vertices reachable from s in the residual network. The standard residual cut, written in terms of row and column masses, yields $\mu(A) - \nu(R(A)) = 1 - f$. The checker verifies this equality directly, so an error in cut extraction is harmless to acceptance. This proves that every finite rational transport input has a certificate of the specified form.

```
proposeTransport(mu, nu, R):
    g, residual := exactMaxFlow(bipartiteNetwork(mu, nu, R))
    r := missing row masses of g
    c := missing column masses of g
    delta := sum(r)
    joint := g if delta = 0 else g + outer(r,c)/delta
    A := empty if delta = 0 else residualReachableLeftVertices
    return joint, delta, A

checkTransport(problem, certificate):
    check joint >= 0, rowSums(joint) = mu, columnSums(joint) = nu
    bad := sum of joint entries outside R
    check bad = claimedDelta = mu(A) - nu(R(A))
```

The dense checker takes $O(|X||Y|)$ rational arithmetic operations. The Edmonds–Karp combinatorial bound is $O(VE^2)$ with $V = |X| + |Y| + 2$; the delivered dense residual representation is intended

for small instances, not high-throughput graph processing. Rational bit complexity is additional. Production search may use a faster exact flow library or a numerical proposal followed by exact repair, but acceptance remains (4).

3.4 Worked examples

Monotone Bernoulli coupling. Let $X \sim \text{Bernoulli}(1/3)$, $Y \sim \text{Bernoulli}(2/3)$, and let $R(x, y)$ mean $x \leq y$. With rows and columns ordered 0, 1, a perfect coupling is

$$\gamma = \begin{pmatrix} 1/3 & 1/3 \\ 0 & 1/3 \end{pmatrix}.$$

The independent product has positive mass at $(1, 0)$, so it cannot prove the same almost-sure relation. The distinction is the choice of a joint law, not an improvement in numerical precision.

A proof that no perfect monotone coupling exists. Reverse the parameters. Rows have masses $(1/3, 2/3)$, columns have masses $(2/3, 1/3)$, and the same order relation is requested. Taking $A = \{1\}$ gives $R(A) = \{1\}$ and $h(A) = 1/3$. The matrix

$$\gamma = \begin{pmatrix} 1/3 & 0 \\ 1/3 & 1/3 \end{pmatrix}$$

has defect exactly $1/3$, so its optimality is settled by three short sum checks. A positive optimal defect is a successful answer, not a failed search.

Two coins versus one binomial sample. Let X be the uniformly distributed pair $(a, b) \in \{0, 1\}^2$, let $Y \sim \text{Binomial}(2, 1/2)$, and require $a + b = Y$. Coupling each pair with its sum gives defect zero. Under independent execution, agreement has probability

$$(1/4)^2 + (1/2)^2 + (1/4)^2 = 3/8,$$

so the defect is $5/8$. This is an explicit distributional-refinement target that independent product execution cannot certify. A runnable example exports both the witness and the independently computed defect.

3.5 Composition with finite probabilistic programs

For finite distributions, define

$$(\mu \gg K)(x') = \sum_x \mu(x) K(x, x').$$

Suppose γ couples μ and ν , and for each pair (x, y) of positive joint mass, $\Gamma_{x,y}$ couples K_x and L_y . Then

$$\gamma'(x', y') = \sum_{x,y} \gamma(x, y) \Gamma_{x,y}(x', y') \tag{6}$$

is a coupling of the two bound distributions. The first marginal follows by summing over y' , substituting the first marginal of each $\Gamma_{x,y}$, and then substituting the first marginal of γ ; the second is symmetric. This finite-sum identity is the initial Lean composition theorem to prove. It corresponds to the existing

PMF.`bind` semantics, whose pointwise formula and associativity law are present at the inspected Mathlib pin [14].

For related state pairs, suppose each proposed transition coupling leaves the relation with probability at most $\varepsilon \in [0, 1]$. Starting in the relation, the probability of leaving it within k steps is at most

$$1 - (1 - \varepsilon)^k \leq k\varepsilon.$$

This follows by conditioning on the first j steps all remaining related and induction on j . Once outside the relation, any coupling of the marginals may be used. The argument concerns a finite synchronous horizon; it does not claim an asynchronous product-program theorem or a general infinite-path measure construction. Those are valuable extensions, but a finite marginal checker should not inherit their names without their proofs.

3.6 A precise next extension: transport costs

The defect problem is a special case of minimizing $\sum_{x,y} c(x,y)\gamma(x,y)$, with cost $c = \mathbf{1}_{\{-R\}}$. A general rational-cost worker can retain the full coupling and replace the Hall subset by potentials u_x, v_y satisfying

$$u_x + v_y \leq c(x,y), \quad \sum_x u_x \mu(x) + \sum_y v_y \nu(y) = \sum_{x,y} c(x,y)\gamma(x,y).$$

The inequality gives a lower bound after multiplication by γ and summation; equality proves optimality. This is a concrete certificate format for min-cost transport, not an implemented feature in this archive.

With a distance cost, a family of witnesses satisfying $\mathbb{E}[d(X', Y')] \leq \kappa d(x, y)$ gives a finite-horizon contraction theorem by iterated expectation. It would let Forge route an approximate relational goal to exact transport duality rather than enumerate candidate algebraic inequalities. The first implementation gate should be the cost certificate and finite composition theorem, before any claim about mixing times.

4 Alternating probabilistic simulation

4.1 A fixed point whose quantifiers matter

A probabilistic implementation can have several enabled actions. To match it, a specification may choose a different action depending on which implementation action was chosen. This cannot be represented by ordinary closure under all pairs of actions.

Let S, T be finite state sets. Each source state s has a nonempty finite set of actions $A(s)$, with transition law $P_{s,a}$ on S . Each target state t has actions $B(t)$ and laws $Q_{t,b}$ on T . Let $R_0 \subseteq S \times T$ be an externally supplied observation compatibility relation. For a relation R , define

$$\mathcal{F}(R) = \left\{ (s, t) \in R_0 : \begin{array}{l} \forall a \in A(s), \exists b \in B(t), \exists \gamma, \\ \gamma \text{ couples } P_{s,a}, Q_{t,b} \text{ and } \text{supp } \gamma \subseteq R \end{array} \right\}. \quad (7)$$

The operator is monotone. A simulation is a post-fixed relation $R \subseteq \mathcal{F}(R)$, and the desired output is the greatest such relation contained in R_0 .

The order of choice is part of the semantics:

$$\forall \text{ source action } a \quad \exists \text{ target action } b \quad \exists \text{ joint transition } \gamma \quad \forall \text{ positive-mass outcomes.}$$

The responder sees the state pair and the source action. It does not see the sampled successor before choosing its action. Swapping those binders changes the refinement contract.

4.2 Positive and negative evidence in one certificate

The proposer starts with R_0 and repeatedly removes a pair that violates (7). Its certificate has two parts.

First, an ordered deletion trace records a removed pair (s, t) , one offending source action a , and for *each* target action b a Hall subset $A_b \subseteq S$ such that

$$P_{s,a}(A_b) > Q_{t,b}(R(A_b)), \quad (8)$$

where R is the live relation immediately *before* this deletion. By Lemma 3.2, no coupling for that target action can be supported by the current relation.

Second, for the final live relation R_* , the certificate lists a target action and a zero-defect coupling for every triple (s, t, a) with $(s, t) \in R_*$ and $a \in A(s)$. The checker verifies complete source-action coverage, rather than accepting a subset of the action table. These couplings establish $R_* \subseteq \mathcal{F}(R_*)$.

Theorem 4.1 (Greatest-simulation certificate). *If every deletion obstruction is valid against its pre-deletion relation, and the final matching table covers all surviving pairs and source actions, then R_* is the greatest simulation contained in R_0 .*

Proof. Let S' be any simulation contained in R_0 . Inductively suppose $S' \subseteq R$ just before a deletion. If the deleted pair (s, t) belonged to S' , the simulation condition for its offending action a would give some target action b and a coupling supported by S' , hence supported by R . That contradicts (8). Thus S' remains a subset after the deletion. At the end, every simulation is contained in R_* . The final matching table proves that R_* itself is a simulation, so it is greatest. $\square$

This is not a negative verdict inferred from saturation stopping. Every removed pair has a local obstruction, and the checker replays the relation changes in order. Conversely, a trace of valid removals without a final complete matching table would establish only exclusion of those pairs, not greatestness of the remaining relation.

4.3 Algorithm, complexity, and incremental improvement

For each live pair and each source action, run the exact transport proposer against target actions until a zero-defect coupling is found. If every target action has positive defect, record the Hall subsets and remove the pair. Restart scanning, and finally generate the matching table for survivors.

```

R := observation-compatible pairs
removed := []
repeat:
  find (s,t) in R and source action a such that:
    for every target action b,
      optimalTransportDefect(P[s,a], Q[t,b], R) > 0
  if no such pair exists: break
  emit (s,t,a, one strict Hall obstruction for every b)
  R := R minus {(s,t)}
emit one supported coupling for every (s,t,a) remaining

```

There are at most $|R_0|$ deletions. For $N = |R_0|$, at most A source actions and B target actions per state, and transport-search cost C , naive repeated scans cost $O(N^2 ABC)$ plus final witness extraction. This is the deliberately transparent prototype. A worklist implementation can index existing couplings

by the relation edges on which they place positive mass. Removing a pair need not invalidate a witness that has zero mass there. Only affected action obligations need a new max-flow computation, which can warm-start from the surviving flow.

Such incremental search must not change the replay format. The replayed obstruction is still checked against the full then-live relation, not the smaller set of edges a cached flow happened to consider. Likewise, deleting states with a heuristic similarity test may guide exploration but cannot justify a deletion record.

A positive matching table has one coupling matrix per surviving pair and source action, so dense certificates may be much larger than the state graph. Sparse nonzero entries are a sensible second format, provided duplicate coordinates are rejected or normalized in one specified way and absent entries mean exactly zero. The delivered version uses dense matrices to keep these issues out of the first checker.

4.4 A quantifier-order example

Take two states 0, 1 on each side and allow two actions everywhere: action 0 moves deterministically to 0, action 1 moves deterministically to 1. Let the base relation be equality. The relation $\{(0, 0), (1, 1)\}$ is a simulation: respond to action a with the same action a . The extracted table therefore contains different target choices for the two source actions.

A procedure searching for one target action that matches *all* source actions would incorrectly reject this example. A procedure that allowed the responder to inspect the sampled outcome before choosing an action could incorrectly accept other examples. The prototype directly tests the first boundary, while the certificate schema fixes the second: each matching record contains a single target action and a full coupling for its law.

There is a further distinction between a mathematical coupling and an implementation protocol. A joint distribution proving refinement does not license changing two separately specified programs to share random bits. Its marginals establish properties of those programs without modifying them. When Forge synthesizes an executable coordinated implementation, access to shared randomness must be a resource present in the source interface.

4.5 The exact meaning of a negative answer

A removed initial pair is not in the greatest simulation defined by (7). It does *not* follow that the programs have unequal trace distributions. Strong simulation is a particular structural relation; trace equivalence and other observational contracts are different questions. The public result should therefore be named something like `ExcludedFromStrongSimulation`, not `ProgramsUnequal`.

Similarly, a base relation R_0 generated from an abstraction must carry its meaning. If it approximates source observations, a proved model simulation transfers only through a sound refinement theorem. Deletion of a pair from an under-approximating model is not a source-level refutation.

4.6 Connection to Leant and Djex

The new synthesis interface is naturally dependent on the local obligation: for every surviving pair and source action, choose a target action and a coupling witness. Leant and Djex already preserve exact selected providers and source-owned typed candidates in their supported fragments [7, 8]. Forge

should reuse that identity discipline when rendering a strategy table, not replace it with equality of printed action names.

The proposed division of labor is concrete. Type-directed synthesis can build the table-shaped term and select source constructors. The simulation worker fills the action and probability data and supplies the behavioral proof. Lean then checks that this term has the original dependent type. This is a new behavioral backend interface; it makes no claim to solve Leant’s separate, still-open Church-indexing or multi-universe synthesis queries.

5 Quantitative liveness: expected termination under every scheduler

5.1 A graph cycle need not prevent probabilistic termination

A retry loop that succeeds with probability $1/4$ on each iteration has a self-loop. There is an infinite path consisting entirely of retries, so no natural-valued function can strictly decrease on every possible transition. Nevertheless, the probability of that infinite path is zero and the expected number of iterations is 4. Forge’s existing pathwise ranking mechanisms cannot certify this merely by searching for more elaborate deterministic ranks. The needed local operator is expectation.

We consider a finite Markov decision process (MDP). Every state s has at least one action; action a induces a rational distribution $P_{s,a}$ over states. The designated terminal states are absorbing in the input itself. The stopping time T is the first time a terminal state is entered, with $T = \infty$ if this never happens. Each nonterminal step has cost 1. A scheduler may depend on the complete finite history and may randomize among available actions. The goal is an upper bound valid for *every* such scheduler.

The terminal and cost conventions matter. A zero-cost loop can have expected cost zero while never terminating. A checker that omits the unit cost from its inequality would prove the wrong property. Likewise, rewriting a nonabsorbing state into a terminal self-loop changes the model unless the source stopping semantics explicitly justifies that transformation.

5.2 A local Bellman certificate

A positive certificate supplies a state region W containing the initial state, closed under every action, and a rational function $V : W \rightarrow \mathbb{Q}_{\geq 0}$. The values at terminal states are zero. For each nonterminal $s \in W$ and *every* available action a , the checker requires

$$V(s) \geq 1 + \sum_t P_{s,a}(t)V(t). \quad (9)$$

Closure ensures that no mass escapes to an unrepresented potential value. The delivered certificate also supplies a deterministic positional policy π attaining equality at every nonterminal state in W .

Theorem 5.1 (Uniform expected-time upper bound). *If (9) holds on a universally closed region containing s_0 , then for every history-dependent randomized scheduler,*

$$\mathbb{E}_{s_0}[T] \leq V(s_0), \quad \mathbb{P}_{s_0}(T = \infty) = 0.$$

For every positive integer N , it also gives $\mathbb{P}_{s_0}(T > N) \leq V(s_0)/N$.

Proof. Condition on a finite execution history that has not yet terminated. The scheduler chooses a distribution over actions. Since (9) holds for each action, it also holds for their convex combination.

Thus the conditional expected change in the potential is at most -1 . Once terminal, both the additional cost and the potential change are zero.

Apply this inequality for exactly N steps to the stopped process. Finite induction and iterated expectation give

$$\mathbb{E}[\min(T, N)] + \mathbb{E}[V(X_{\min(T, N)})] \leq V(s_0). \quad (10)$$

Dropping the nonnegative second term and increasing N proves the expected-time bound by monotone convergence. A positive mass at $T = \infty$ would force $\mathbb{E}[\min(T, N)]$ to grow without bound, so that mass is zero. Finally $N\mathbb{P}(T > N) \leq \mathbb{E}[T]$ gives the tail estimate. $\square$

The proof uses a finite-horizon inequality before taking a limit. A Lean implementation need not begin by invoking an optional-stopping theorem whose integrability side conditions it has not yet established. It can first define finite execution-tree expectations by recursion and prove (10). The measure-theoretic interpretation is a separate, reusable bridge.

Theorem 5.2 (Exact worst-case expectation). *If a positional policy π makes (9) an equality at every nonterminal state of W , then*

$$\sup_{\text{all schedulers } \sigma} \mathbb{E}_{s_0}^{\sigma}[T] = \mathbb{E}_{s_0}^{\pi}[T] = V(s_0).$$

Proof. For policy π , the finite telescoping argument yields equality in (10). The upper-bound theorem already establishes almost-sure termination under this policy. Since W is finite, V is bounded by a finite constant M , and the remaining-potential term is at most $M\mathbb{P}(T > N)$, which tends to zero. The equality therefore converges to $\mathbb{E}^{\pi}[T] = V(s_0)$. The upper-bound theorem bounds all other schedulers by the same number. $\square$

A certificate intended only to prove a budget could omit the tight policy and accept any valid supersolution. The prototype uses the stronger exact-optimum contract to make differential testing against exhaustive policy evaluation possible. A budget query compares $V(s_0)$ to the original budget only after this certificate is checked.

5.3 A finite certificate of nontermination

A negative certificate supplies a nonempty set U of nonterminal states and one action $\rho(s)$ at each $s \in U$, such that

$$\text{supp}(P_{s, \rho(s)}) \subseteq U. \quad (11)$$

It also supplies a finite path from s_0 to U , labeling each edge with a legal action and a strictly positive transition probability. The checker multiplies these probabilities and verifies the claimed positive product α .

Proposition 5.3 (Reachable-trap soundness). *A valid path-and-trap certificate establishes a scheduler with $\mathbb{P}(T = \infty) \geq \alpha > 0$, hence infinite expected termination time.*

Proof. Use a history-dependent scheduler that tries the listed path and, when that path has been followed, switches to ρ in U . The probability of following the entire finite path is α . Conditional on that event, all subsequent transitions stay in U with probability one and no terminal state is reached. A nonnegative random variable that is infinite on a positive-probability event has infinite expectation. $\square$

The witness scheduler may use finite memory during the prefix; it need not be a single stationary table compatible with every occurrence along the path. The universal upper-bound goal allows all history-dependent schedulers, so this is a valid counterexample. An unreachable nonterminal trap is not a counterexample from s_0 , which is why the path cannot be omitted.

5.4 A complete algorithm on the stated finite fragment

First compute W , the states reachable from s_0 under the union of all positive-probability action edges. Let $U_0 = W \setminus \text{Terminal}$ and iterate

$$U_{k+1} = \{s \in U_k : \exists a, \text{supp}(P_{s,a}) \subseteq U_k\}. \quad (12)$$

The sequence decreases and stabilizes after at most $|W|$ strict rounds. If its limit is nonempty, select one staying action at each surviving state and recover a reachability path from the initial search. This yields Proposition 5.3.

Suppose instead that the limit is empty. Assign each nonterminal state the round in which it is removed. At removal, *every* action has some positive probability of moving to a terminal state or a state removed earlier. Let $\eta > 0$ be the smallest positive transition probability in the reachable finite model. Under any scheduler and after any history, there is probability at least $\eta^{|W|}$ of following rank-decreasing successors to termination within $|W|$ steps. Repeating the argument in blocks gives a uniform geometric tail and a finite expected-time bound. This establishes that all stationary policies are proper before any policy matrix is inverted.

Now perform maximizing policy iteration. For a deterministic stationary policy π , let Q_π be its transition matrix restricted to reachable nonterminal states. Evaluate it exactly by

$$(I - Q_\pi)v_\pi = \mathbf{1}. \quad (13)$$

At each state, compute $1 + P_{s,a} \cdot v_\pi$ for all actions. Switch only when an action is strictly greater than the current value. Re-evaluate and repeat. The prototype uses exact Gauss–Jordan elimination, not rounded matrix inversion.

Lemma 5.4 (Policy improvement after the trap test). *When the trap fixed point is empty, every policy evaluation is nonsingular, and a strictly improving switch cannot cause a policy cycle.*

Proof. Properness and the uniform geometric tail imply $(I - Q_\pi)^{-1} = \sum_{k \geq 0} Q_\pi^k$, a finite-entry nonnegative matrix. For a switched policy π' , let $d = \mathbf{1} + Q_{\pi'}v_\pi - v_\pi$. The switching rule makes $d \geq 0$ and at least one component positive. Then

$$v_{\pi'} - v_\pi = (I - Q_{\pi'})^{-1}d \geq 0.$$

Because the inverse includes its identity summand, at least one component strictly increases. A return to an earlier policy would return to its unique value vector, contradicting monotone strict progress. There are finitely many stationary policies, so iteration terminates. $\square$

At termination, the current policy has equality in its evaluation equations and no action exceeds its value. The vector and policy therefore satisfy both Bellman certificate checks.

Theorem 5.5 (Finite-fragment decision result). *For a finite total MDP with exact rational probabilities and absorbing terminal states, the algorithm terminates with either an exact worst-case expected-time certificate or a reachable nonterminal trap certificate. The former proves finite expected time under every scheduler; the latter refutes that property.*

Proof. The finite trap iteration terminates. A nonempty result gives the negative certificate. An empty result establishes uniform properness, after which Lemma 5.4 guarantees termination of policy iteration. The final local equations give the positive certificate and its correctness by Theorems 5.1 and 5.2. $\square$

The dichotomy relies essentially on finiteness. It does not assert that every almost-surely terminating infinite-state program has finite expected time. Nor does failure to find a symbolic polynomial potential refute termination.

5.5 Worked scheduler examples

Geometric retry with an adversary. There are two states: retry r and terminal z . At r , an adversary chooses between success probabilities $1/2$ and $1/4$; unsuccessful trials return to r . Let $V(r) = 4$ and $V(z) = 0$. The two Bellman right-hand sides are

$$1 + \frac{1}{2} \cdot 4 = 3, \quad 1 + \frac{3}{4} \cdot 4 = 4.$$

Thus every scheduler has expected duration at most 4, and always selecting the slower action attains 4. No graph edge has been removed: the retry self-loop is present in both the model and the proof.

A good policy does not prove a universal bound. At r , one action terminates immediately and another stays at r with probability one. An existential controller can terminate in one step, but the universal scheduler statement is false. The trap $\{r\}$ with the staying action and the empty prefix has $\alpha = 1$. The prototype checks that it returns this negative outcome rather than the expected duration of the convenient terminating policy.

An unreachable trap is irrelevant. Add a separate absorbing nonterminal state u with no path from s_0 . The reachability slice removes it before the trap test. A hand-written negative certificate naming u without a valid path is rejected. This demonstrates a necessary source of root dependence: changing the initial state can change the theorem even when every transition table remains identical.

5.6 A symbolic extension that reuses existing Forge machinery

For an infinite-state probabilistic loop, let the update be $x' = F(x, \xi)$ with a finitely supported random choice ξ . Given a polynomial potential V , the local drift obligation is

$$V(x) - \sum_{\xi} p_{\xi} V(F(x, \xi)) \geq \varepsilon > 0$$

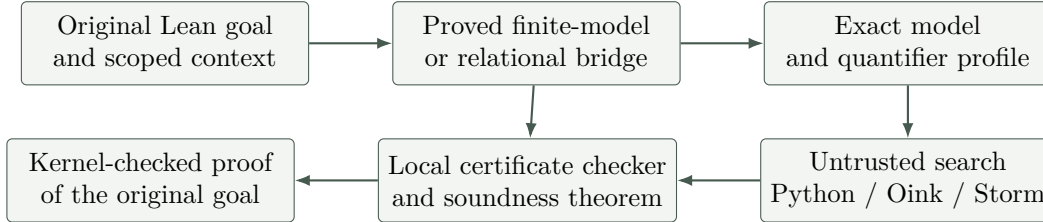
on the nonterminal invariant, together with $V \geq 0$. Forge's existing exact polynomial and cone workers can discharge this obligation once the expected substitution is reified. They should not demand the stronger and often false condition $V(x) - V(F(x, \xi)) \geq \varepsilon$ for each outcome.

For example, a walk on positive integers moves down with probability p and up with probability $q = 1 - p$, with $p > q$, and stops at zero. Taking $V(n) = n/(p - q)$ gives expected drift exactly 1 at every positive state. The finite-horizon telescoping argument gives $\mathbb{E}[T] \leq V(n)$ without a pointwise rank. This is a designed composition with existing Forge certificates, not a symbolic-search feature executed by the delivered finite-state prototype. State-dependent probabilities require their own nonnegativity and normalization proofs; clearing a denominator without proving its sign is not an acceptable translation.

6 Integration into Forge and Lean

6.1 Extend the workers, not the architecture

Forge already separates problem representation, proposed evidence, checking, and publication into a source proof. Its `CertificateSpec` contract contains a denotation, a Boolean check, and a theorem that accepted certificates imply that denotation [5]. None of the new algorithms requires a replacement scheduler or a second general-purpose proof search framework. They should become workers under that existing interface.



The diagram is an intended integration architecture. Only the exact-model, Python-search, and Python-checker portions were executed here. The bridge and kernel arrows remain implementation obligations, not boxes made trustworthy by being drawn next to one another.

6.2 A small neutral problem language

Use four first-order input forms matching the delivered dataclasses. Each has an explicit semantic version, supplied by the eventual enclosing protocol. The Python research format discriminates by `kind`; it does not yet implement Forge’s full versioned worker transport.

Problem	Required source data	Semantic choices to preserve
Parity arena	Owners, priorities, total legal-successor relation, queried initial state or full-region query	Max recurrent priority; strategy observations; legal adversary moves
Transport	Two normalized rational distributions and a relation matrix	Full marginals; defect event; equality or budget requested
Simulation	Source and target action distributions; observation relation	$\forall a \exists b$ before sampling; actions as source-owned identities
Expected time	Actions, absorbing terminals, initial state	Unit cost before termination; supremum over all schedulers

Do not erase a terminal predicate, an action owner, or a probabilistic choice into an uninterpreted label and then infer its semantics from the solver’s answer. In particular, a parameterized probability can be reified as a rational only after the source term has been proved equal to that rational. Substituting a decimal approximation is a different theorem.

The source language can begin with finite enumerated states and commands **pure**, finite rational **sample**, **bind**, and an explicit finite action choice. Reification produces a transition table *and* a proof of its meaning, by compositional equations for these commands. This is much smaller than translating arbitrary Lean programs or arbitrary recursive functions to a probabilistic model.

6.3 Source bridges as the critical new obligations

Enumeration. For a finite source state type α , use a proved equivalence $e : \alpha \simeq \text{Fin } n$, or a supplied finite cover with the appropriate semantic theorem. A list of distinct sample states is not a proof that the state space has been covered. For dependent actions, enumeration is per source state, not a single untyped action number reused across unrelated states.

Transition interpretation. For a probabilistic command $K : \alpha \rightarrow \text{PMF } \alpha$, the bridge must establish the equality between its point masses and the rational table lifted to nonnegative extended reals. Nonnegativity and total mass one are proved before the lift. At the pinned Mathlib revision, `Mathlib.Probability.ProbabilityMassFunction.Monad` defines `PMF.pure`, `PMF.bind`, and `PMF.bind_apply`; these are actual inspected declarations, not speculative API names [14]. The corresponding `Basic`, `Constructions`, and `Integrals` modules are present in the same directory. The first bridge should stay on finite types so that its essential algebra reduces to finite sums.

Strategies. Reconstruct a policy as a dependent function selecting a legal action, not as a list of integers whose legality is assumed. A winning-region theorem should state its universal environment quantifier explicitly. Any available-information restriction on the strategy is part of the source type. The prototype has perfect state information; it is not a solver for partially observable games.

Path and stopping semantics. The first expected-time theorem can use finite stopped execution trees. Define $H_0(h) = 0$ for each history h . If its last state is terminal, define $H_{N+1}(h) = 0$; otherwise take one plus the finite weighted average of H_N over the next action and state. Induction gives $H_N(h) \leq V(\text{last}(h))$ under the Bellman inequalities, even when the action weights depend on all of h . The theorem equating the increasing limit of these quantities with $\mathbb{E}[T]$ can then use the established probability library. A result about H_{100} alone is not an unbounded termination proof.

6.4 Proposed module boundaries

The following names are a proposed addition, not files already present in Forge:

```
Forge/Finite/Distribution.lean
  normalized finite rational tables; interpretation and exact sums
Forge/Relational/Transport.lean
  coupling checker; marginal soundness; Hall lower bound; bind rule
Forge/Game/Parity.lean
  strategy closure; threshold-rank checker; infinite-play theorem
Forge/Probabilistic/Simulation.lean
  supported matches; deletion-trace soundness; greatestness theorem
Forge/Probabilistic/Bellman.lean
  finite-horizon bound; exact tight-policy result; trap refutation
Forge/Reify/FinitePrograms.lean
  source enumerations, command semantics, action identities
```

Keep the mathematical theorems separate from reflection. For example, prove that *any* matrix with the specified marginals is a coupling, then prove that the Boolean matrix checker establishes those equations. Likewise, prove Theorem 5.1 about a semantic transition kernel once, rather than generating a new optional-stopping proof for each certificate.

The emitted source proof should consist of a checked certificate application followed by the source bridge. It must not rerun search during ordinary recompilation. Large data can be represented as shared arrays or a compact certificate declaration, but a compact representation still needs a theorem connecting its decoded entries to the denotation.

6.5 Result types should name their authority

Each lane needs more than a Boolean success flag. The following are suggested semantic outcomes, translated into Forge’s existing publication mechanism:

```

Parity:
  WinningStrategy(player, region, checked witness)
Transport:
  OptimalDefect(value, coupling, Hall witness)
Simulation:
  GreatestSimulation(relation, matches, exclusion trace)
Expected time:
  ExactWorstExpectation(value, tight policy, Bellman witness)
  PositiveNonterminationProbability(prefix, trap, lower bound)
Common:
  Unsupported(fragment reason) | Unknown(resource reason)

```

A transport optimum above zero refutes perfect R -couplability, not equality in distribution for every relation. A simulation exclusion refutes membership in that specified simulation. An exact worst expectation above a budget supplies a policy violating the budget even when all policies still terminate. An MDP trap proves nontermination with positive probability, not merely a large expected duration.

For a source proposition G , only the existing certificate-soundness theorem and a proved bridge may construct a `CheckedOutcome.closed` proof. Model-level negative evidence with no reverse bridge is a diagnostic or a new model theorem, not a proof of $\neg G$. This reuses Forge’s distinction between authority and search guidance without repeating its general security design.

6.6 Concrete library choices

Component	Concrete choice	Integration contract
Current exact prototype	Python standard library: <code>fractions</code> , <code>collections</code> , <code>dataclasses</code> , <code>unittest</code>	All four searches, checkers, and small oracles run with site packages disabled. No native solver or array library is required.
Parity scale-up	Oink [10]	Read its winning strategy, re-extract threshold ranks, and run the local checker. Solver preprocessing requires state-map reconstruction before source publication.
Temporal front end	Spot [12]	Useful for proposing automata and game products. A separate theorem must relate the automaton to the source formula; strategy checking alone does not verify the translation.

Component	Concrete choice	Integration contract
Probabilistic scale-up	Storm / stormpy [13]	Propose policy and model structure for finite MDPs. Re-evaluate the policy rationally and check every Bellman inequality, rather than trusting a reported optimum.
Lean probability semantics	Mathlib ProbabilityMassFunction modules at the pinned commit [14]	Prove a finite rational-table interpretation and compositional <code>bind</code> bridge. Infinite-horizon results require their own semantic connection.
Symbolic drift obligations	Existing Forge polynomial/cone workers [2]	Apply to exact expected substitutions and nonnegativity conditions; do not rebuild the existing nonlinear backend.
Typed policy terms	Leant/Djex source-owned synthesis surfaces [7, 8]	Preserve dependent action types and exact provider choices; attach the new behavioral certificates without broadening the synthesis fragment’s negative claims.

As verified on 15 September 2026, Storm’s official site lists version 1.14.0 (released 20 August 2026), and Spot lists version 2.16 (released 3 September 2026) [13, 12]. These are candidate integration versions, not packages installed or executed in this work. The Python results do not depend on either version. Oink is cited as an algorithmic library choice without an unverified claim about its latest release.

6.7 How to use `grind` without treating it as a model checker

The surrounding proof can still use `grind` for equalities, bounds on indices, case normalization, and local logical consequences [9]. For instance, a source invariant may reduce a parameterized transition probability to one of a few rational values. A certificate worker should be requested after that normalization has produced a proved finite problem, not before.

Conversely, the worker can return a proved quantitative fact to the local solver: an optimal defect is zero, a bound is at most the user’s budget, or an initial state belongs to a winning region. This is genuine theory cooperation. It does not require exposing the whole max-flow residual network or the entire policy iteration history to `grind`.

7 Implementation and experimental evidence

7.1 Execution conditions and units

The full run used Python 3.13.5 on `Linux-6.18.44-x86_64-with-glibc2.41`, with seed 20260915. It was invoked with `python -S`, which disables site packages. Every generated problem and its accepted certificate is retained in `results/full/corpus.json`. The run conditions and counts appear in `results/full/report.json`. A smaller smoke run is retained separately and is not added to the full-run totals.

Fourteen focused regression tests also passed with no failures or errors. Mutation counts differ because a second semantic mutation is generated only when its precondition exists, such as a positive transport

Worker	Problems	Independent comparisons	Invalid mutations rejected
Parity arenas	884	884	1,128
Transport	400	400	723
Expected time	180	180	180
Simulation	100	100	86

Table 4: Executed finite-model tests. Each problem produced an accepted Python certificate agreeing with its oracle. Rows are different workloads, not a pooled theorem-proving success rate.

defect or an applicable strict parity-rank edge. They are counts of deliberately invalid certificates, not of uniformly random bit flips. No tactic baseline, proof elaboration time, kernel-checking time, or user-theorem solve rate was measured.

7.2 Parity: exhaustive small arenas and random larger ones

The exhaustive part includes every total arena on one or two vertices, with both owner choices and priorities from $\{0, 1, 2, 3\}$. There are 8 one-vertex arenas and 576 two-vertex arenas, hence 584 in total. A further 300 seeded arenas have 3–8 vertices, one or two successors per vertex, random owners, and priorities from $\{0, \dots, 5\}$.

The oracle does not call Zielonka or check threshold ranks. It enumerates positional strategies of both players and follows the resulting deterministic run to its eventual cycle. A player wins a start vertex when it has a strategy winning against every positional strategy of the opponent. Finite positional determinacy makes this an independent small-arena decision computation. Every returned winning partition agreed with this oracle.

The mutations remove a vertex from the required partition, or deliberately violate one applicable strict threshold inequality. The semantic regressions also test a one-time favorable priority, an infinite favorable cycle, and an omitted adversarial edge. These controls target the certificate theorem, not merely the solver’s ability to reproduce its own partition.

7.3 Transport: exact optima and an actual ablation

The 400 instances use marginal dimensions independently chosen from 1–6. Unnormalized integer weights are chosen from 0–4, with an all-zero vector repaired by assigning positive weight to one coordinate, then normalized exactly. Relation entries are seeded Boolean choices.

The oracle enumerates every subset A of the left support and computes $\max_A h(A)$ directly. It never constructs a residual network. Each max-flow proposal’s exact defect agreed with this maximum. Of the 400 optima, 77 were zero and 323 were positive. Both are valid completed answers because the certificate proves the optimum, not merely feasibility.

For the same input, the experiment also evaluates the defect of the independent product $\mu \otimes \nu$. On **211 of 400 instances**, the synthesized coupling has strictly smaller defect. On the others it ties; it is never worse. This is a genuine paired ablation of a weaker relational method, not a comparison with another Lean tactic. It establishes that optimizing correlation matters in these generated cases; it does not establish a population success rate for source-level verification problems.

One mutation corrupts a joint entry, violating a marginal. When the optimum is positive, another erases the Hall subset, creating a nonzero primal–dual gap. All are rejected. Separate regressions reject

missing probability mass, noncanonical rational numbers, and independent-product data masquerading as a zero-defect correlation certificate.

7.4 Expected time: a structurally different oracle

The 180 MDPs have 2–4 states, one or two actions per nonterminal state, a single absorbing terminal state, and initial state 0. Marginals use the same exact integer-weight generator. In every even-indexed case, each nonterminal action is modified to put at least probability 1/2 directly on the terminal state; these are controlled positive fixtures embedded in the random workload. Odd-indexed cases have no such guarantee.

The oracle enumerates stationary deterministic policies. For each policy it uses graph reachability to detect a reachable closed nonterminal class. When none exists, it computes the finite expected duration by determinant expansion and Cramer’s rule on the reachable nonterminal subsystem. This deliberately avoids the proposer’s Gauss–Jordan routine. It then takes the worst policy value, or reports nontermination when an appropriate policy exists.

The proposer returned **164 exact finite worst-expectation certificates**. The remaining **16 cases** returned positive-probability nontermination certificates. Every outcome and every finite initial-state value agreed with the oracle. The negative certificate’s positive path probability was separately rechecked. Mutating a positive result’s initial potential to zero or a negative result’s probability lower bound to zero was rejected in every case.

The tiny sizes are intentional: exhaustive policy enumeration and determinant expansion provide a transparent differential check, not a competitive large-MDP solver. A follow-up performance evaluation should retain these oracles for small cases while separately studying larger models; it should not replace exact small-case validation with agreement between two wrappers around the same numerical solver.

7.5 Simulation: enumerate relations, not elimination traces

The 100 pairs of systems have 1–2 states on the left and 1–3 on the right, one or two actions per state, and a seeded Boolean base relation. The oracle enumerates every subrelation of that base and checks its post-fixed property using exhaustive Hall-subset tests. The union of all post-fixed relations is then the greatest simulation. This is a different computation from the proposer’s ordered pair-removal loop.

The resulting greatest relations agreed on every problem. Across this workload, 77 pair deletions and 81 surviving pairs were recorded; these are *pair counts*, not problem counts or solved theorems. Invalid tests erase a strict Hall witness or remove a required action match when such data exists. The quantifier-order regression explicitly requires different target actions for different source actions.

7.6 Replay without search or test oracles

The command

```
cd prototype
python -S replay.py ../results/full/corpus.json \
  --receipt ../results/full/replay.json
```

was executed. The replay process installs an import hook that rejects imports of `forge_ap.search` and the entire `tests` namespace, then imports only representations and the checker. It accepted all

stored records: 884 parity, 400 transport, 180 MDP, and 100 simulation records. The receipt records `search_imports = FORBIDDEN` and `site_packages_enabled = false`.

This is an operational separation test, not a formal proof of software independence. The search, checker, and oracle still share input dataclasses, normalization rules, Python integers, and rational arithmetic. A defect in a shared representation or an incorrect mathematical model could affect every computation. The independence claim is specifically that the checking algorithm does not invoke the search algorithms or the finite test oracles.

7.7 What the measured timings do and do not mean

The recorded full test run took approximately 0.414 seconds and the replay approximately 0.070 seconds in this environment. These figures are retained as raw execution metadata, not as performance evidence. They combine tiny synthetic cases, local arithmetic, serialization, and test bookkeeping; they are not repeated measurements and have no Lean baseline. No speedup or throughput claim should be derived from their ratio.

The algorithm counters are similarly diagnostic: the parity proposer made 2,884 recursive calls, the transport workload used 1,219 augmentations, the MDP workload used 383 trap-iteration passes and 231 policy evaluations, and the simulation workload invoked transport search 386 times. These counters have different units and are deliberately not combined into a scalar “proof effort” score.

7.8 Remaining implementation weaknesses

The research parser rejects duplicate keys, floating-point and nonfinite JSON numbers, noncanonical rational pairs, out-of-range indices, and malformed array shapes. Its bounds are not a hostile-input security design. In particular, it bounds individual arrays but does not enforce one global budget for all matrix cells or intermediate rational arithmetic. Resource-limit rejections must be distinguished from malformed input at a production adapter; neither implies that a mathematical proposition is false.

The code is small and intentionally dense in places. Its searches are not interruptible Forge workers; they do not yet receive cancellation tokens, subprocess resource quotas, or snapshot handles. The Python checker is not formally verified, and the Lean specimens do not implement a decoder or source reifier. These are concrete remaining tasks, not hidden assumptions behind the passing tests.

8 Implementation sequence and acceptance gates

8.1 First ship a probability bridge, not four new tactic names

Forge’s existing recommendation to obtain one narrow end-to-end checked lane remains sensible [6]. The proposed additions should not interrupt that baseline work. Within this extension, the smallest useful first milestone is a *positive finite transport certificate connected to an actual source distribution*. It needs finite sums, nonnegativity, and an explicit source equality; it does not need infinite plays, a greatest fixed point, or probability measures on infinite paths.

The first source theorem should be the two-coins/binomial relation from Section 3, written using genuine probability distributions. The input marginals must be derived from the source expressions by checked `pure/bind` equations. The accepted theorem must mention those original expressions. A theorem saying merely that a literal rational matrix has matching row sums does not close this milestone.

8.2 Six concrete delivery gates

Gate 1: finite rational reflection. Implement validated tables, exact sum checking, and a theorem that accepted tables denote probability distributions. Establish an equivalence between the source finite type and its index set. Test zero mass, duplicate indices, incorrect normalization, and a source enumeration missing one constructor. Acceptance requires both a successful source theorem and rejection of the same certificate against an altered marginal.

Gate 2: transport and composition. Prove the full-marginal checker sound, then the Hall lower bound, then the finite `bind` composition rule. Check a positive example, a positive-defect obstruction, and a two-step composition whose second coupling depends on the first state pair. The last test prevents an implementation from supporting only constant transition kernels while claiming compositional reasoning.

Gate 3: simulation witnesses. Implement source-owned dependent action tables and replay of the ordered deletion trace. Prove greatestness, not only post-fixedness. The acceptance corpus must include the $\forall a \exists b$ example, a removed initial pair, an empty base relation, and a deliberately incomplete target-action obstruction list. A simulation exclusion must not be rendered as a source trace-inequality proof.

Gate 4: expected duration. Prove the finite-horizon Bellman bound, the tight-policy result, and the path/trap result. Connect the finite-horizon definition to the chosen source expectation semantics. Required source tests include the two-action retry example, an unreachable trap, a reachable trap, an initially terminal state, and a finite exact expectation that exceeds a proposed user budget. Checking only stationary schedulers is insufficient for the stated universal theorem.

Gate 5: parity. Prove strategy closure and the threshold-rank theorem for infinite sequences. Transfer an explicit finite source arena to its table. Check an infinite even cycle, a favorable priority appearing only once, two adversarial strategies, and a source-state renaming. Temporal formula translation is a separate gate; accepting an automaton as input is not an LTL synthesis implementation.

Gate 6: scale and composition. Add untrusted Oink and Storm adapters, sparse encodings, cancellation, and incremental search only after the semantic gates pass. Compare the adapters on identical reified source problems and identical checker versions. An adapter must be removable without changing the meaning of retained proofs. Add a composite example where a relational simulation supplies a state invariant used by a Bellman worker, or a synthesized finite strategy changes the nondeterministic model subsequently analyzed for expected cost.

8.3 A useful composite target

Consider an implementation that retries a randomized operation using one of several equivalent sampling routines. The specification uses a simpler sampler. A transport certificate can establish each routine's distributional relation; a simulation certificate can justify matching every implementation choice by a specification transition; and a Bellman potential on the resulting source model can establish a uniform expected number of retries.

The composition must record exactly which property transfers. Simulation by itself need not preserve a particular cost unless the base relation and the transition theorem include that cost. For expected duration, a relation that matches one implementation step with one specification step and preserves the terminal predicate is a suitable initial target. Stuttering or asynchronous simulation would require an additional accounting potential. This is a concrete place to combine the workers without silently converting one theorem into another.

8.4 Evaluation after Lean integration

The first empirical comparison should not ask whether `grind` is a parity solver. It should ask whether adding a worker closes natural source-level goals that the same Forge controller, premise set, and local tactics leave open. Hold constant the source bridge, retrieved lemmas, proof budget, and definitions. For each family, compare the full worker with an intentionally restricted alternative on the same goals: independent-product couplings, non-alternating action matching, or pointwise instead of expected drift. Record failure as UNKNOWN unless the restricted method has a checked obstruction for its own exact fragment.

Measure search time, source reification time, certificate size, elaboration time, kernel checking time, and ordinary recompilation separately. Include false goals, malformed certificates, and equivalent source programs with renamed constructors. Keep theorem families disjoint between tuning and measurement. A tiny generated model accepted by a Python checker is not a replacement for any row of this source-level evaluation.

8.5 Research directions after the implemented fragment

Three expansions have especially clear certificate paths. First, min-cost couplings and dual potentials can replace Boolean relations by quantitative distances without changing the marginal checker. Second, symbolic drift can reuse Forge’s existing exact polynomial machinery after a proved expected substitution. Third, probabilistic games can combine adversarial strategy choices with stochastic transitions; local inequalities become nested minimum/maximum expectation conditions, and qualitative recurrence requires its own end-component analysis. The third extension is *not* implemented by merely combining the current parity and MDP parsers.

Another practical direction is policy compression. A finite table can be synthesized into a small source-level decision tree using Leant/Djex or Forge’s existing witness facilities, then proved extensionally equal to the certified table. This can improve the readability and executable size of extracted controllers without forcing the game solver to search directly in a higher-order term language. Compression is an optional post-pass: failure leaves the original checked table usable.

8.6 Assessment

The proposed increase in capability is specific. Forge gains finite witnesses for infinite adversarial behavior, exact relational correlations, quantified matching of probabilistic actions, and expected termination in the presence of cycles. These are proof shapes its existing universal-closure and pathwise-rank workers do not supply by themselves.

The four implementations demonstrate the certificate computations and reject important semantic corruptions. The formalization obligation is equally specific: prove the local checker theorems and connect exact finite models to original Lean expressions. The delivered evidence supports the former

as a mathematical design and an executed Python prototype, not yet as a Lean implementation. A successful next contribution should make that missing bridge small, checked, and reusable rather than enlarge the planner’s unverified surface.

A Artifact guide and reproducibility

A.1 Archive layout

article/	
forge-alternation-probability.tex	Main editable source
forge-alternation-probability.pdf	Rendered article
sections/*.tex, references.tex	Complete included sources
build.sh, build.ps1	pdfLaTeX build commands
prototype/	
forge_ap/models.py	Exact validated representations
forge_ap/search.py	Four untrusted proposers
forge_ap/check.py	Local checkers, no solver imports
tests/oracles.py	Independent tiny-model computations
tests/test_regressions.py	Fourteen focused regression tests
run_experiments.py	Deterministic differential suite
replay.py	Search-forbidden certificate replay
examples.py	Worked examples and certificates
results/	
full/	Full corpus, report, replay receipt
smoke/	Separate initial smoke run
examples/	Worked-example records and summary
lean/	
RankTelescoping.lean	Uncompiled arithmetic lowering lemma
CouplingAlgebra.lean	Uncompiled finite marginal lemmas
README.md	Exact status and remaining obligations
docs/	
STATUS.md, AUDIT.md	Evidence and non-duplication boundary
sources.json	Repository pins and reviewed paths
README.md, LICENSE	

The own-authored code and article use MIT-0. No third-party source trees, published paper PDFs, or font files are included. The archive does not modify or replace the user’s repository.

A.2 Run the experiments

The code uses Python 3.10 or newer syntax; the recorded execution used 3.13.5. It requires no installed packages. From the archive root:

```
cd prototype
python -S -m unittest discover -s tests -v
python -S run_experiments.py --output ../reproduced-results/full
python -S replay.py ../results/full/corpus.json
python -S examples.py --output ../reproduced-results/examples
```

The experiment harness refuses to overwrite an existing report in its chosen output directory. Use a

new path for each run. If an experiment fails, its partial corpus, a failure status, and a traceback are retained. The recorded smoke run is a separate workload and must not be pooled with the full run.

Search-free replay proves only that this Python checker accepted these records without importing the search or oracle modules. It does not establish that an arbitrary untrusted execution environment would enforce the import hook, or that the parser is secure against hostile resource exhaustion.

A.3 Build the PDF

The article uses stock pdfLaTeX packages, Latin Modern, and TikZ. No system font installation, shell escape, bibliography program, or network access is needed. From `article/`, run `bash build.sh`, or run the following command three times:

```
pdflatex -interaction=nonstopmode -halt-on-error \
forge-alternation-probability.tex
```

A PowerShell equivalent is included as `build.ps1`. The main source includes all sections and its bibliography; no external repository checkout is required.

A.4 Reviewed source snapshots

Source	Exact snapshot / reviewed surface
Forge	c98e47c5f804e92880fc1d0e378c1b95832b685c. README; docs/STATUS.md; docs/IDEAS-FROM-LEANT-DJEX.md; article/sections/B-provenance.tex; article/sections/12-conclusion.tex; lean/Forge/Design/Contracts.lean; recursive tree metadata.
Leant	6bf05ad78c467989e68290f2d08bbbed40802d485. README capability summary, source-owned synthesis description, and current scope limitations.
Djex	e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. README engine and typed-provider capability summary.
Mathlib	1cf325a0cf67aca2b04d76b5380ff6a9e410aefa. ProbabilityMassFunction directory and the definitions/theorems in <code>Monad.lean</code> .

The Forge comparison uses the canonical inventory and provenance, not a claim that every historical file was read. Leant and Djex were consulted for integration fit, not comprehensively re-audited. Sources are pinned where possible because a proposal that is genuinely additional at one snapshot may be merged into the repository later.

A.5 Certificate fields and checker costs

Family	Principal certificate data	Local verification
Parity	Partition; one own move per region vertex; rank arrays indexed by unfavorable priority	Region closure and threshold-edge inequalities; no recursive game solving

Family	Principal certificate data	Local verification
Transport	Full rational joint matrix; exact bad mass; Hall subset	Marginals, nonnegative entries, defect sum, primal–dual equality
Simulation	Ordered pair removals with one Hall obstruction per target action; complete survivor matching table	Replay relation deletions; check strict obstructions and every surviving action coupling
Expected time	Universally closed region, nonnegative potential, tight policy; or reachable trap plus positive prefix	All-action Bellman inequalities and policy equalities; or positive path product and selected-action trap closure

Rationals use canonical integer pairs `[numerator, denominator]` with a positive denominator. JSON floats, duplicate keys, nonfinite values, Boolean values used as integers, and unreduced fractions are rejected. State sets are sorted distinct index lists. All matrix dimensions are checked against the external problem representation. These are reproducibility conventions, not proofs that the implementation has no parser bugs.

A.6 The included Lean specimens

`RankTelescoping.lean` gives a natural-number prefix-sum telescoping lemma: if $r(n+1)+w(n) \leq r(n)$, then $r(n) + \sum_{k < n} w(k) \leq r(0)$. It isolates the arithmetic used after a parity play has entered a fixed threshold tail. It does not formalize the arena, infinite recurrence, or the threshold checker.

`CouplingAlgebra.lean` states a rational finite coupling structure and proves the direct marginal consequences used in a future finite-sum bridge. It does not implement the Boolean transport checker, Hall duality, or a PMF interpretation. Both files are supplied with proof scripts but were **not compiled**. No axiom inventory is reported; the included `#print axioms` commands are instructions for a subsequent actual Lean execution, not recorded output.

References

- [1] Vladimir Reshetnikov et al. *Forge: README and merged design*, snapshot `c98e47c5f804e92880fc1d0e378c1b95832b685c`, inspected 15 September 2026. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/README.md>.
- [2] Forge. *Implementation status*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/STATUS.md>.
- [3] Forge. *Provenance: what came from where*, Appendix B of the merged article. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/B-provenance.tex>.
- [4] Forge. *Ideas from Leant and Djex*, same snapshot. <https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/docs/IDEAS-FROM-LEANT-DJEX.md>.

- [5] Forge. *Certificate and publication contracts*, same snapshot.
<https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/lean/Forge/Design/Contracts.lean>.
- [6] Forge. *Assessment and recommended starting point*, section 12.
<https://github.com/VladimirReshetnikov/Forge/blob/c98e47c5f804e92880fc1d0e378c1b95832b685c/article/sections/12-conclusion.tex>.
- [7] Vladimir Reshetnikov et al. *Leant: a Djex-based synthesis REPL for Lean 4*. README at 6bf05ad78c467989e68290f2d08bbbed40802d485. <https://github.com/VladimirReshetnikov/Leant/blob/6bf05ad78c467989e68290f2d08bbbed40802d485/README.md>.
- [8] Vladimir Reshetnikov et al. *Djex*. README at e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5. <https://github.com/VladimirReshetnikov/Djex/blob/e8778f4ebd63e1f9b9b410fa4de8d14a8a04c9e5/README.md>.
- [9] Lean development team. *Lean Language Reference: The grind tactic*. Official documentation, inspected 15 September 2026.
<https://lean-lang.org/doc/reference/latest/The--grind--tactic/>.
- [10] Tom van Dijk. *Oink: an Implementation and Evaluation of Modern Parity Game Solvers*. TACAS 2018. DOI: 10.1007/978-3-319-89960-2_16. <https://arxiv.org/abs/1801.03859>. Software: <https://github.com/trolando/oink>.
- [11] Gilles Barthe, Benjamin Grégoire, Justin Hsu, and Pierre-Yves Strub. *Coupling proofs are probabilistic product programs*. POPL 2017. DOI: 10.1145/3009837.3009896. <https://arxiv.org/abs/1607.03455>.
- [12] Spot developers. *Spot: a platform for LTL and ω -automata manipulation*. Official project documentation, inspected 15 September 2026. <https://spot.lre.epita.fr/>.
- [13] Storm developers. *Storm: A Modern Probabilistic Model Checker*. Official project documentation, inspected 15 September 2026. <https://www.stormchecker.org/>.
- [14] Mathlib contributors. *Monad Operations for Probability Mass Functions*. Source at 1cf325a0cf67aca2b04d76b5380ff6a9e410aefa; definitions and finite-sum bridge targets inspected directly.
<https://github.com/leanprover-community/mathlib4/blob/1cf325a0cf67aca2b04d76b5380ff6a9e410aefa/Mathlib/Probability/ProbabilityMassFunction/Monad.lean>.